

FORMATION INGÉNIEUR

du Conservatoire National des Arts et Métiers,
spécialité **Informatique et Multimédia**
(en partenariat avec l'Université de Toulon)

PROMOTION 2024-2025, 4^e ANNÉE, SEMESTRE 4

Cahier des Charges Technique

« Application web d'orientation des publics, Pôle Innovation CNAM PACA »

Projet :	R4, Pilotage d'un projet numérique d'orientation
Entreprise :	Pôle Innovation, CNAM PACA
Apprenti / chef de projet :	DEMOISSON Arthur
Maître d'Apprentissage :	AUCHET Anne-Flore
Tuteur Académique :	ROCHE Benoît
Document de référence :	CCF R4 v1.0 (28 avril 2026)
Version du document :	1.0, validée en revue technique du 04 mai 2026
Date :	4 mai 2026

Objet du document :

Le présent Cahier des Charges Technique (CCT) traduit en choix d'architecture, en composants logiciels et en exigences mesurables le besoin métier formalisé dans le Cahier des Charges Fonctionnel (CCF v1.0). Il documente l'architecture applicative retenue (PHP vanilla / MySQL, sans framework), le modèle de données, les mécanismes de sécurité (authentification, sessions durcies, jeton CSRF, *rate limiting*, en-têtes HTTP), la stratégie de déploiement (preprod / prod sur infrastructure mutualisée), les indicateurs de performance et leur mode de mesure, le registre de conformité RGPD, ainsi que les procédures d'exploitation, de sauvegarde, de monitoring et de reprise d'activité. Il sert de référence d'ingénierie pour la mise en production, la maintenance et toute évolution ultérieure du produit.

1. Présentation du document

1.1. Objet

Le présent Cahier des Charges Technique (ci-après « CCT ») traduit les exigences fonctionnelles et non-fonctionnelles formalisées dans le Cahier des Charges Fonctionnel (CCF v1.0 du 28 avril 2026) en choix d'architecture, en composants logiciels, en mécanismes de sécurité et en indicateurs de performance mesurables. Il constitue le document d'ingénierie de référence pour la mise en production, la maintenance et toute évolution ultérieure de l'application web d'orientation portée par le Pôle Innovation du CNAM PACA.

Là où le CCF répond à la question « *que doit faire la solution ?* », le CCT répond à la question « *comment la solution est-elle construite, déployée, exploitée et mesurée ?* ». Aucun choix technique d'envergure ne peut être pris sans figurer dans ce document ou dans une révision ultérieure validée selon le même processus que la version 1.0.

1.2. Glossaire technique

Terme	Définition
API	<i>Application Programming Interface</i> , interface de programmation exposée par le serveur pour le client web.
CSP	<i>Content Security Policy</i> , en-tête HTTP qui restreint les sources autorisées de scripts, styles, images.
CSRF	<i>Cross-Site Request Forgery</i> , attaque exploitant la session d'un utilisateur authentifié pour soumettre une requête non désirée.
DDL / DML	<i>Data Definition / Manipulation Language</i> , sous-ensembles de SQL (création de schéma vs manipulation de données).
HSTS	<i>HTTP Strict Transport Security</i> , en-tête forçant le navigateur à utiliser HTTPS pour une durée donnée.
HTTP/2	Version 2 du protocole HTTP, multiplexage de plusieurs requêtes sur une seule connexion TCP/TLS.
InnoDB	Moteur de stockage transactionnel de MySQL ; supporte les contraintes d'intégrité référentielle.
PDO	<i>PHP Data Objects</i> , couche d'abstraction d'accès aux bases de données en PHP, supportant les requêtes paramétrées.
RPO	<i>Recovery Point Objective</i> , quantité maximale de données pouvant être perdues lors d'un incident.
RTO	<i>Recovery Time Objective</i> , durée maximale d'indisponibilité acceptable d'un service.
SLA	<i>Service Level Agreement</i> , engagement de niveau de service.
SLO	<i>Service Level Objective</i> , objectif interne de niveau de service, plus strict que le SLA.
SPA	<i>Single Page Application</i> , application web monopage rendue côté client.
TLS	<i>Transport Layer Security</i> , protocole de chiffrement en transit (anciennement SSL).
TTFB	<i>Time To First Byte</i> , délai entre la requête HTTP et la réception du premier octet de réponse.

Terme	Définition
UTC	<i>Universal Time Coordinated</i> , temps universel coordonné, fuseau de référence des horodatages.
vhost	<i>Virtual Host</i> , configuration nginx déclarant un service web pour un nom de domaine donné.
XSS	<i>Cross-Site Scripting</i> , injection de code script malveillant dans une page consultée par d'autres utilisateurs.

1.3. Versions et révisions

Version	Date	Auteur	Modifications
1.0	2026-05-04	A. Demoisson	Version contractuelle validée en revue technique (architecture, modèle de données, sécurité, RGPD, KPI, plan d'évolution).

1.4. Documents de référence

Référence	Document
CCF-R4	Cahier des Charges Fonctionnel R4, version 1.0 du 28 avril 2026.
PLANNING	<code>PLANNING.md</code> versionné dans le dépôt Git du projet, état d'avancement vivant.
INSTALL	<code>INSTALL.md</code> , procédure d'installation et d'exploitation pas à pas.
MEETINGS	<code>MEETINGS.md</code> , comitologie et template de compte rendu.
SCOPE	<code>docs/scope-formations.md</code> , périmètre des 18 formations CNAM PACA retenues.
RGPD-EU	Règlement (UE) 2016/679, Règlement Général sur la Protection des Données.
WCAG-2.1	<i>Web Content Accessibility Guidelines 2.1</i> niveau AA, référentiel d'accessibilité numérique du W3C.
OWASP-T10	<i>OWASP Top 10</i> (2021), référentiel des dix risques de sécurité applicative web les plus critiques.

2. Cadre technique général

2.1. Choix technique général

La pile retenue est **PHP vanilla + MySQL + nginx**, sans framework côté serveur, sans framework côté client, sans gestionnaire de paquets. Quatre raisons.

D'abord, le périmètre fonctionnel (test binaire, calcul de score, CRUD admin, export CSV) ne mobilise rien que PHP natif et SQL standard ne fassent en quelques fichiers. Un framework apporterait surtout de la complexité.

Ensuite, la maintenabilité par les équipes internes est une exigence du commanditaire. Un développeur PHP/MySQL classique reprend la solution sans formation préalable.

L'hébergement mutualisé n'autorise pas l'installation de runtimes additionnels ni d'outillages côté serveur (Composer, npm, build pipeline). PHP 8.5 et MySQL 8.4 sont disponibles ; les fonctionnalités modernes du langage (types stricts, `match`, attributs, `enum`) sont utilisées directement.

Enfin, l'objectif sobriété (≤ 200 ko transférés par page, fixé au CCF) est incompatible avec une pile JavaScript moderne : React, Vue ou Angular livrent à eux seuls plusieurs centaines de kilo-octets avant la moindre ligne d'application.

2.2. Pile technologique retenue

Couche	Composant	Version cible	Justification
Système d'exploitation	Ubuntu Server LTS	24.04 ou supérieure	Distribution standard, mises à jour de sécurité long terme garanties par Canonical.
Serveur HTTP / proxy	nginx	1.26 ou supérieure	Faible empreinte mémoire, configuration déclarative, support natif de HTTP/2 et de la terminaison TLS.
Runtime applicatif	PHP-FPM	8.5	Version courante au démarrage du projet, supportée jusqu'à 2027 ; types stricts, <i>match</i> , attributs, <i>enum</i> .
Base de données	MySQL	8.4 LTS	Compatible avec l'instance existante du CNAM PACA, support des contraintes <code>CHECK</code> , JSON natif disponible si besoin futur.
Couche d'accès BDD	PHP PDO	natif	Couche d'abstraction standard, requêtes paramétrées, prévention des injections SQL au niveau du langage.
Front-end	HTML5, CSS3, JavaScript ES2020	navigateurs ≥ 2 ans	Compatible avec le parc des smartphones du public cible sans transpilation.
Certificat TLS	Let's Encrypt via <i>certbot</i>	renouvellement automatique	Coût nul, automatisation prouvée, durée de vie 90 jours avec renouvellement à T-30.
Vers ionnement	Git + GitHub	dernière	Hébergement du code, intégration native avec le suivi de projet (issues, releases, kanban).
Intégration continue	GitHub Actions	dernière	Déclenchement à chaque <i>push</i> , gratuit pour dépôts

Couche	Composant	Version cible	Justification
			privés du commanditaire.

2.3. Décisions documentées

Quatre décisions doivent être tracées ici car elles conditionnent l'évolutivité et la maintenance.

Pas de framework PHP (cf. §2.1). Contrepartie : un peu plus de code à écrire à la main pour certaines briques génériques (routing, validation). Compensé par la lisibilité : pas de magie d'autoloading, routage explicite dans `api/index.php`, dépendances déclarées en haut de chaque fichier.

Pas de gestionnaire de paquets. Toute dépendance externe doit être copiée manuellement dans le dépôt. Cette friction est volontaire : elle empêche l'ajout impulsif de bibliothèques tierces et oblige à instruire chaque introduction. À ce jour, zéro dépendance externe.

Pas d'envoi de mail côté serveur. La prise de contact passe par un lien `mailto:` ouvert dans le client mail natif du candidat. Avantages : pas de dépendance à un serveur SMTP, pas de secret SMTP à gérer, pas de surface d'attaque (relais ouvert, bounces, listes noires), candidat maître de l'identité émettrice. Contrepartie acceptée : pas d'archivage centralisé des contacts.

Pas de cookies de suivi tiers. Aucune solution analytique tierce (Google Analytics, Matomo hors site). Les indicateurs de §8 reposent uniquement sur les logs applicatifs et les requêtes SQL, ce qui simplifie la conformité RGPD.

3. Architecture applicative

3.1. Vue d'ensemble

L'application suit une architecture classique en **trois couches** :

1. Une **couche de présentation** côté client, composée de pages HTML statiques et de modules JavaScript chargés à la demande. Aucune logique métier ne réside côté client : tous les calculs de score, toutes les vérifications d'autorisation, toutes les mutations de données passent par l'API serveur.
2. Une **couche d'API HTTP** côté serveur, exposée sous le chemin `/api/`, qui dispatche les requêtes vers les contrôleurs en fonction du couple (méthode, route).
3. Une **couche de persistance** reposant sur MySQL via PDO, encapsulée par des classes *Model* qui exposent une API typée au reste du code.

Les trois couches communiquent par contrat explicite : le client appelle des routes documentées qui retournent du JSON typé ; les contrôleurs orchestrent et délèguent aux modèles ; les modèles retournent des structures associatives stables.

3.2. Arborescence du dépôt

L'organisation des répertoires reflète la séparation des responsabilités. Tous les chemins sont relatifs à la racine du dépôt.

```

r4/
├─ index.html          Point d'entrée du parcours utilisateur (test).
├─ login.html          Connexion / inscription.
├─ account.html        Espace personnel d'un utilisateur connecté.
├─ admin.html          Tableau de bord administrateur.
├─ 404.html, 500.html  Pages d'erreur personnalisées.
├─ style.css           Feuilles de style globales (mobile-first).
├─ app.js              Logique du test (parcours, état, rendu).
├─ account.js, admin.js Logique des espaces respectifs.
├─ ping.php            Endpoint de monitoring : retourne 200 OK
                        sans accès BDD ; cible des sondes externes.
├─ api/
│   └─ index.php       Routeur HTTP unique (front controller).
│   └─ .htaccess       Réécriture vers index.php (Apache), référence ;
                        équivalent nginx en production (cf. §6).
├─ app/
│   └─ Core/
│       └─ Database.php Connexion PDO singleton.
│       └─ Response.php Helpers de réponse JSON normalisée.
│   └─ Models/
│       └─ User.php     Comptes, rôles, mots de passe hachés.
│       └─ Question.php Questions et options du test.
│       └─ Formation.php Formations + recommandation.
│       └─ Survey.php   Réponses persistées et historique.
│   └─ Controllers/
│       └─ AuthController.php Inscription, connexion, déconnexion.
│       └─ QuestionController.php Lecture publique des questions et formations.
│       └─ RecommendController.php Calcul de recommandation.
│       └─ AnswerController.php Persistance des réponses.
│       └─ StatsController.php Statistiques admin.
│       └─ AdminController.php CRUD questions/formations + export CSV.
├─ config/
│   └─ config.php       Lecture de l'environnement (.env ou variables système).
├─ database/
│   └─ schema.sql       Schéma initial.
│   └─ migration_v2.sql Refonte questions/formations CRUD.
│   └─ migration_v3_cnam.sql Périmètre CNAM PACA + niveau RNCP.
│   └─ migration_v4_questions.sql 30 questions thématiques + soft skills.
│   └─ migration_v5_quick.sql Mode test rapide (10 questions).
│   └─ seed_v2.sql, seed_users.sql Jeux de données.
│   └─ install.sql      Bundle complet schema + migrations + seed.
├─ docs/
│   └─ scope-formations.md 18 formations CNAM PACA retenues.
│   └─ ccf-r4.md           Cahier des Charges Fonctionnel.
│   └─ cct-r4.md           Présent document.
├─ PLANNING.md, INSTALL.md, MEETINGS.md, README.md
└─ .github/workflows/lint.yml Intégration continue.

```

3.3. Routeur HTTP

Le routeur unique réside dans `api/index.php`. Il s'agit d'un *front controller* léger d'environ cent lignes qui prend en entrée la méthode HTTP et le chemin réécrit (capture par `_route` dans la réécriture nginx), applique deux *middlewares* successifs, puis dispatche vers le contrôleur via une expression `match`.

Les deux *middlewares* appliqués sont :

1. **Garde d'autorisation administrateur** sur les routes préfixées `admin/*`. La session doit contenir un `user_id` non vide et un rôle valant `admin`. À défaut, la requête se conclut par une réponse 403 sans atteindre le contrôleur.
2. **Garde CSRF** sur les méthodes mutantes (`POST`, `PUT`, `DELETE`) des routes listées explicitement. Le client doit fournir l'en-tête `X-CSRF-Token` correspondant au jeton stocké en session ; la comparaison est effectuée par `hash_equals()` (résistant aux attaques par mesure de temps).

Le dispatch effectif repose sur `match ([$method, $route])` : chaque route est listée explicitement, sans table de routes générique. Cette approche, plus verbeuse qu'une introspection automatique, garantit qu'aucune route involontaire n'est exposée et qu'une lecture du fichier suffit à connaître la totalité de la surface d'API.

3.4. Couche modèle

Les classes du dossier `app/Models/` jouent le rôle de couche d'accès aux données. Elles n'utilisent pas d'ORM : chaque méthode embarque la requête SQL paramétrée correspondante. Cette transparence facilite la revue de sécurité (toute requête est relisible immédiatement) et l'optimisation par index ciblés.

Modèle	Responsabilité	Méthodes principales
User	Création de compte, vérification de mot de passe, lecture par id ou nom.	<code>create()</code> , <code>findByUsername()</code> , <code>verifyPassword()</code> .
Question	Lecture des questions actives et de leurs options, support du mode rapide.	<code>listActive(?bool \$quickOnly)</code> , <code>findById(int \$id)</code> .
Formation	Lecture du catalogue, calcul de recommandation pondéré par les scores.	<code>listAccessible(int \$userLevel)</code> , <code>recommend(array \$answers, ?int \$userLevel)</code> .
Survey	Persistance d'une session de réponses, lecture de l'historique d'un utilisateur.	<code>save(int \$userId, array \$answers, ?int \$userLevel)</code> , <code>listForUser(int \$userId)</code> .

Toutes les requêtes utilisent des *prepared statements* PDO. Aucune chaîne provenant du client ne franchit la concaténation SQL. La connexion PDO est instanciée en mode `ERRMODE_EXCEPTION`, fait remonter les erreurs SQL comme exceptions captées par le routeur (cf. §3.3), et ne fait jamais fuiter de message technique vers le client.

3.5. Couche présentation côté client

Le client est composé de quatre pages HTML autonomes et de leurs modules JavaScript associés. Chaque module est encapsulé dans une *Immediately Invoked Function Expression* (IIFE) pour éviter toute pollution du *scope* global ; aucune variable n'est exposée sur `window`.

Les feuilles de style sont mobile-first (les règles de base ciblent les petits écrans, les *media queries* élargissent ensuite). Les *design tokens* (couleurs, espacements, rayons, durées de transition) sont déclarés sous forme de variables CSS dans `:root`, ce qui permet une refonte graphique par modification d'un seul bloc. Le mode sombre est pris en charge automatiquement via `prefers-color-scheme: dark`.

L'invalidation du cache CDN s'effectue par paramètre de requête versionné sur les liens vers `style.css` et `app.js` (`?v=YYYYMMDDhhmmss`). À chaque livraison qui modifie ces fichiers, l'horodatage est mis à jour. Les ressources sont alors récupérées par les visiteurs même si Cloudflare ou un cache navigateur intermédiaire en détenait une version antérieure.

3.6. Format des échanges API

Toutes les réponses API sont au format JSON, encodage UTF-8, avec le `Content-Type: application/json` positionné dès l'entrée du routeur. La structure est normalisée par les helpers `Response::json($payload)` et `Response::error($message, $statusCode)` :

- Les réponses de succès retournent un objet JSON dont la forme dépend de la route, sans enveloppe additionnelle (pas de champ `success: true` superflu).
- Les réponses d'erreur retournent `{ "error": "<message>" }` avec le code HTTP adapté (400, 401, 403, 404, 500).
- Aucune information technique de bas niveau (chemin de fichier, *stack trace*, message PDO) n'est jamais incluse dans la réponse au client. Les détails sont enregistrés dans le journal serveur consultable par l'équipe technique.

4. Modèle de données

4.1. Schéma relationnel

La base de données `r4_survey` repose sur **sept tables** liées par contraintes d'intégrité référentielle. Toutes les tables utilisent le moteur InnoDB (transactions, contraintes), le jeu de caractères `utf8mb4` et la collation `utf8mb4_unicode_ci`, ce qui garantit le support intégral d'Unicode (caractères accentués, signes typographiques, emojis).

Table	Rôle	Cardinalité indicative
<code>users</code>	Comptes utilisateurs (visiteurs inscrits + administrateurs).	dizaines à centaines.
<code>questions</code>	Questions du test, avec drapeau <code>quick</code> pour le mode 10 questions.	trentaines.
<code>question_options</code>	Options de réponse rattachées à une question (deux par question).	soixantaines.
<code>formations</code>	Catalogue des formations avec niveau RNCP et url officielle.	dix-huit en v1.
<code>formation_scores</code>	Pondération entre une option de réponse et une formation.	une centaine.
<code>survey_responses</code>	Une ligne par session de test complète d'un utilisateur connecté.	croissance continue.
<code>response_answers</code>	Détail des réponses d'une session (texte de la question figé).	10 à 30 lignes par session.

Les sessions anonymes ne génèrent aucune ligne en base : le calcul de recommandation est effectué *à la volée* par `Formation::recommend()` sans persistance, la liste de recommandation est rendue au client puis perdue.

4.2. Conventions de nommage

Les noms de tables sont au pluriel et en *snake_case*. Les clés primaires sont uniformément nommées `id` et typées `INT UNSIGNED AUTO_INCREMENT`. Les clés étrangères suivent le motif `<table>_id` (par exemple `user_id`, `response_id`). Les colonnes d'horodatage portent les suffixes `_at` et utilisent `TIMESTAMP DEFAULT CURRENT_TIMESTAMP`. Les colonnes booléennes sont stockées en `TINYINT(1)` avec valeur par défaut explicite. Les contraintes nommées préfixées `fk_` ou `idx_` permettent un repérage rapide en cas d'analyse de plan d'exécution.

4.3. Intégrité référentielle et cascades

Les contraintes de clé étrangère sont déclarées explicitement et utilisent `ON DELETE CASCADE` sur les liens « parent → enfant ». Concrètement, supprimer un utilisateur purge en cascade ses `survey_responses` et leurs `response_answers`. Cette politique est volontaire : elle simplifie l'application du droit à l'effacement RGPD à une seule requête `DELETE` sur la table `users`.

Sur les tables de référence (`formations`, `questions`), les suppressions sont en pratique remplacées par une désactivation logique (drapeau `active`) : on n'efface jamais une question utilisée dans des réponses passées, sous peine de perdre la traçabilité historique. Les vues d'administration filtrent par défaut sur `active = 1` et permettent l'affichage des entrées inactives sur demande explicite.

4.4. Stockage du texte des questions et des réponses

La table `response_answers` stocke à la fois `question_key` et `question_text`, ainsi que `chosen_value` et `chosen_label`. Cette duplication apparente est délibérée : si un Responsable de formation reformule une question existante, les sessions passées doivent continuer d'afficher la formulation au moment où elles ont été données. Sans cette copie, l'historique perdrait son sens dès la première édition.

4.5. Migrations versionnées

Le schéma a évolué en cinq versions successives, chacune matérialisée par un fichier SQL distinct conservé dans `database/`. Le fichier `install.sql` consolide schéma + migrations + jeux de données dans un ordre exécutable d'un seul trait sur une base vierge. Les migrations sont appliquées dans l'ordre numérique sur les bases existantes ; chaque migration est idempotente lorsque c'est possible (`CREATE TABLE IF NOT EXISTS`, `ADD COLUMN IF NOT EXISTS` quand supporté).

Migration	Contenu
<code>schema.sql</code>	Création initiale des trois tables transactionnelles <code>users</code> , <code>survey_responses</code> , <code>response_answers</code> .
<code>migration_v2.sql</code>	Création des quatre tables de référence (<code>questions</code> , <code>question_options</code> , <code>formations</code> , <code>formation_scores</code>) et mise en place du CRUD admin associé.
<code>migration_v3_cnam.sql</code>	Ajout de <code>formations.level</code> et <code>survey_responses.user_level</code> , remplacement des cinq formations de démonstration par les dix-huit du périmètre CNAM PACA, refonte du scoring.
<code>migration_v4_questions.sql</code>	Passage de 10 à 30 questions thématiques (couverture domaine par domaine + soft skills), 60 options, 124 lignes de scoring rebâties pour les 18 formations.
<code>migration_v5_quick.sql</code>	Ajout du drapeau <code>questions.quick</code> et marquage des dix questions retenues pour le mode rapide.

Toutes les migrations sont validées en preprod avant application en production. La procédure d'application en production figure dans `INSTALL.md` .

4.6. Index et plans d'exécution

Les index ont été ajoutés en réponse aux requêtes effectivement exécutées par l'application, et non en spéculation.

- `users.username` et `users.email` : `UNIQUE` , supportent la connexion et la vérification d'unicité à l'inscription.
- `survey_responses.user_id` : index simple, supporte la liste d'historique d'un utilisateur (`WHERE user_id = ?`).
- `response_answers.response_id` : index simple, supporte la jointure de détail d'une session.
- `response_answers.question_key` : index simple, supporte les statistiques par question (`GROUP BY question_key`).
- `formation_scores (option_id, formation_id)` : clé composite, supporte le calcul de recommandation.
- `questions.active` , `questions.quick` : index courts, supportent les filtres de chargement de questions actives et du mode rapide.

Les plans d'exécution ont été contrôlés sur des jeux de données représentatifs (~1000 sessions, 30 000 lignes de réponses) avec `EXPLAIN` . Aucune requête de l'application ne déclenche de *full table scan* sur les tables transactionnelles.

5. Sécurité applicative

5.1. Modèle de menace

Le modèle de menace retenu cible explicitement les risques applicatifs du référentiel **OWASP Top 10 (2021)**, complétés des spécificités du contexte (hébergement mutualisé, base mutualisée). Les huit catégories ci-dessous représentent les surfaces effectivement exposées par l'application.

Risque OWASP	Pertinence	Mitigation effective dans le projet
A01, Broken Access Control	Élevée	<code>Middleware admin/*</code> dans le routeur, <code>\$_SESSION['role']</code> vérifié, pas de référence directe à l'identifiant dans les routes utilisateur.
A02, Cryptographic Failures	Élevée	Mots de passe stockés via <code>password_hash()</code> (bcrypt) ; cookies de session <code>Secure</code> en HTTPS ; tokens CSRF générés par <code>random_bytes(32)</code> .
A03, Injection	Critique	Requêtes 100 % paramétrées via PDO ; aucune concaténation SQL avec entrée client ; <code>.htaccess</code> interdit l'exécution PHP en dehors de <code>api/</code> .
A04, Insecure Design	Moyenne	Périmètre minimal (pas d'envoi mail, pas de cookies tiers, pas de bibliothèques transitives) ; gardes explicites en début de routeur.
A05, Security Misconfiguration	Élevée	<code>display_errors=0</code> en prod, <code>log_errors=1</code> , en-têtes de sécurité (<code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , CSP) au niveau nginx.
A06, Vulnerable Components	Faible	Aucune dépendance externe applicative ; PHP/MySQL/nginx maintenus par les paquets du système d'exploitation.
A07, Identification and Auth Failures	Élevée	<i>Rate limiting</i> sur <code>/api/auth</code> (compteur de tentatives en session) ; sessions régénérées à la connexion (<code>session_regenerate_id(true)</code>).
A08, Software and Data Integrity	Moyenne	CI sur chaque <i>push</i> (<code>php -l</code> + garde-fous de motifs interdits) ; pas de mises à jour automatiques côté code applicatif.
A09, Security Logging Failures	Moyenne	<code>error_log</code> PHP redirigé vers fichier en prod ; logs d'accès nginx archivés ; logs CSRF/auth sont enregistrés en cas d'échec.
A10, Server-Side Request Forgery	Sans objet	L'application n'effectue aucune requête sortante depuis le serveur.

5.2. Authentification et gestion des sessions

L'authentification repose sur les comptes locaux stockés dans la table `users`. Aucun annuaire externe (LDAP, SSO d'établissement) n'est intégré dans le périmètre v1, l'absence d'API d'annuaire ouverte aux applications du Pôle Innovation ayant été constatée lors du cadrage. Cette dette fonctionnelle est documentée et susceptible d'être traitée dans une itération ultérieure.

Les mots de passe sont hachés à la création de compte avec `password_hash($password, PASSWORD_DEFAULT)`, ce qui sélectionne automatiquement l'algorithme courant recommandé par PHP (bcrypt à coût 12 au moment de la rédaction). La vérification utilise `password_verify()`, qui inclut une comparaison résistante aux attaques par mesure de temps. La logique applicative ne manipule jamais de mot de passe en clair en dehors du moment immédiat de la requête.

Les sessions PHP sont durcies dès le démarrage du routeur :

- `session.cookie_httponly = 1` interdit l'accès au cookie de session depuis JavaScript, neutralisant l'exfiltration par XSS.
- `session.cookie_samesite = Lax` permet les liens entrants (par exemple depuis une page CNAM) sans exposer le cookie aux requêtes POST cross-origin.
- `session.cookie_secure = 1` est positionné lorsque la requête arrive en HTTPS, ce qui interdit la transmission du cookie en clair.
- À la connexion réussie, `session_regenerate_id(true)` est appelé pour invalider l'identifiant pré-authentification (mitigation des attaques par fixation de session).

5.3. Protection CSRF

Toute requête mutante (`POST`, `PUT`, `DELETE`) doit présenter un jeton CSRF dans l'en-tête `X-CSRF-Token`. Le jeton est généré paresseusement à la première lecture via `bin2hex(random_bytes(32))`, soit 64 caractères hexadécimaux issus du générateur cryptographiquement sûr du système. Le jeton est stocké en session côté serveur et restitué au client par la route `GET /api/csrf`, lue au démarrage de chaque page. La comparaison côté serveur s'effectue par `hash_equals()`, fonction conçue pour être insensible au timing.

L'absence de jeton, sa non-correspondance ou la non-existence du jeton serveur entraîne une réponse 403 immédiate sans atteindre le contrôleur. La route `GET /api/csrf` elle-même n'est pas protégée (lecture seule, aucun effet de bord).

5.4. Limitation de débit (rate limiting)

La route `POST /api/auth` (connexion / inscription / déconnexion) est protégée par un compteur de tentatives stocké en session. Au-delà de **cinq tentatives consécutives échouées dans une fenêtre de cinq minutes**, la route renvoie 429 (*Too Many Requests*) jusqu'à expiration de la fenêtre.

Cette limitation niveau session protège efficacement contre les *brute force* opportunistes depuis un même navigateur. Une attaque distribuée depuis plusieurs IPs nécessiterait une couche complémentaire au

niveau nginx (`limit_req`), prévue dans la procédure d'exploitation et activable sans modification applicative.

5.5. En-têtes de sécurité HTTP

La configuration nginx ajoute systématiquement les en-têtes suivants à toutes les réponses servies sous le domaine de production :

En-tête	Valeur	Effet
Strict-Transport-Security	<code>max-age=31536000; includeSubDomains</code>	Force le navigateur à utiliser HTTPS pour un an, y compris pour les sous-domaines.
X-Content-Type-Options	<code>nosniff</code>	Désactive l'inférence de type MIME par le navigateur (mitigation XSS par fichier mal typé).
X-Frame-Options	<code>SAMEORIGIN</code>	Interdit le <i>framing</i> de l'application depuis un autre domaine (mitigation <i>clickjacking</i>).
Referrer-Policy	<code>strict-origin-when-cross-origin</code>	Réduit l'exposition de l'URL d'origine sur les requêtes sortantes.
Content-Security-Policy	<code>default-src 'self'; img-src 'self' data:; style-src 'self' 'unsafe-inline'</code>	Restreint les sources autorisées de scripts, images, styles.
Permissions-Policy	<code>geolocation=(), microphone=(), camera=()</code>	Désactive explicitement les API navigateurs non utilisées par l'application.

Le `'unsafe-inline'` accordé aux styles est temporaire et lié à quelques attributs `style=""` résiduels dans les fichiers HTML d'erreur ; un *backlog item* de Phase 5 prévoit de les externaliser pour permettre une CSP plus stricte.

5.6. Protection des dossiers sensibles

Les répertoires `app/`, `config/`, `database/` ne contiennent aucun fichier qui doive être servi via HTTP. Leur exposition est interdite à deux niveaux :

1. **Au niveau nginx** par un bloc `location ~ ^/(app|config|database)/ { return 403; }` dans le vhost de production.
2. **Au niveau Apache de référence** par un `.htaccess` racine déclarant `Deny from all` sur ces dossiers, à titre de défense en profondeur si le projet venait à être déployé dans un environnement

Apache.

Le fichier `database/setup.php` (script d'initialisation) est exécutable une seule fois lors de l'installation, puis supprimé conformément à la procédure d' `INSTALL.md` . Sa présence en production déclencherait un point de vérification explicite lors de la revue de mise en service.

5.7. Gestion des secrets

Les secrets (mot de passe BDD, clé de signature éventuelle, identifiants d'administrateur de bootstrap) résident dans un fichier `.env` à la racine du dépôt, jamais commité (`.gitignore`). Lu au démarrage par `config/config.php` qui peuple `$_ENV` . En production ou pré-production, les variables peuvent alternativement être définies au niveau système (Apache `SetEnv` , systemd `EnvironmentFile` , ou variable `nginx-PHP-FPM`), recommandé dès qu'un déploiement automatisé est en place.

L'inventaire des secrets est tenu hors-dépôt dans le coffre `pass` du chef de projet ; une copie de procédure de rotation des secrets est consignée dans `INSTALL.md` .

6. Infrastructure et déploiement

6.1. Environnements

Trois environnements distincts sont définis et ont chacun un rôle clair dans le cycle de vie.

Environnement	URL	Rôle	Données
Développement	<code>http://localhost:8080</code>	Itération rapide, tests manuels par le développeur.	Base de développement, jeu de données seed minimal.
Pré-production	<code>https://r4-preprod.ohvenus.fr</code>	Validation des évolutions par le chef de projet et le MA avant production.	Base distincte de la production, snapshot anonymisé.
Production	<code>https://r4.ohvenus.fr</code>	Service ouvert au public final.	Base réelle, données de production.

Les environnements pré-production et production sont hébergés sur le même serveur (Ubuntu Server, IP `149.202.61.166`), via deux *virtual hosts* nginx distincts. Cette mutualisation est acceptable dans le contexte de la version 1 ; un redécoupage sur deux machines distinctes pourra être engagé si la charge le justifie ou si les obligations de séparation s'imposent à plus long terme.

6.2. Configuration nginx

Chaque vhost est configuré selon le même squelette :

- Bloc d'écoute en HTTP (port 80) qui redirige systématiquement vers HTTPS via une réponse 301.
- Bloc d'écoute en HTTPS (port 443) qui sert la racine du projet, déclare le certificat TLS Let's Encrypt et applique les en-têtes de sécurité de la section 5.5.
- Bloc `location` racine qui réécrit les requêtes `/api/...` vers `api/index.php?_route=...` (équivalent de la `RewriteRule` Apache de référence dans le `.htaccess`).
- Blocs `location` interdisant l'accès aux dossiers sensibles (`app`, `config`, `database`) et au fichier `.env`.
- Bloc `location ~ \.php$` passant les requêtes PHP au socket PHP-FPM (`unix:/run/php/php8.5-fpm.sock`), avec timeouts adaptés aux exports CSV (jusqu'à trente secondes).

Une configuration de référence complète figure en `INSTALL.md` ; toute évolution structurelle du vhost est validée par revue avant application.

6.3. Certificats TLS

Les certificats sont émis par Let's Encrypt via le client *certbot* installé en mode autonome sur le serveur. Le renouvellement est planifié par l'unité systemd `certbot.timer`, qui tente le renouvellement deux fois par jour à partir du trentième jour avant expiration. Le rechargement de nginx s'effectue automatiquement via le *deploy hook* `--deploy-hook 'systemctl reload nginx'`.

Une supervision externe minimale (cron quotidien interrogeant `openssl s_client`) vérifie que la date de validité est supérieure à quinze jours et déclenche une alerte par mail au chef de projet en cas de dérive. Le statut courant des certificats est consigné dans la fiche projet du vault personnel.

6.4. Procédure de déploiement

Le déploiement suit un processus en quatre étapes, exécutées manuellement par le chef de projet à ce stade du projet (volume d'évolutions modeste, automatisation prématurée).

1. **Validation en pré-production**, la branche de fonctionnalité est mergée sur `preprod`, le serveur de pré-production est resynchronisé (`git pull`), les migrations SQL pertinentes sont appliquées sur la base de pré-production, le parcours utilisateur clé est rejoué en navigateur.
2. **Revue technique**, relecture de la PR par le chef de projet sur GitHub, vérification des résultats de la CI (`php -l`, motifs interdits, ancres HTML/CSS).
3. **Merge en production**, la PR est mergée sur `master`, le *checkout* du serveur est resynchronisé sur `master`. Le cache CDN est invalidé par bump de la chaîne `?v=...` sur les liens vers `style.css` et `app.js`.
4. **Smoke test post-déploiement**, la page d'accueil prod (`https://r4.ohvenus.fr`) répond 200 ; la route `GET /api/csrf` retourne un jeton ; un test rapide de bout en bout est effectué.

Toute migration de schéma destructive (par exemple suppression de colonne, renommage) est précédée d'une sauvegarde manuelle de la base via `mysqldump`. Les migrations purement additives (ajout de colonne avec valeur par défaut, ajout de table) sont appliquées sans sauvegarde supplémentaire en s'appuyant sur la sauvegarde mutualisée quotidienne (cf. §6.5).

6.5. Sauvegarde et reprise d'activité

Les sauvegardes de la base MySQL reposent sur le dispositif mutualisé de l'infrastructure : un *dump* logique quotidien de toutes les bases du serveur, conservé sept jours en local et trente jours sur stockage distant. Cette politique se traduit en objectifs de service :

- **RPO (Recovery Point Objective) = 24 heures.** En cas de perte totale de la base, les données saisies depuis la dernière sauvegarde sont perdues.
- **RTO (Recovery Time Objective) = 4 heures.** Délai indicatif entre l'incident déclaré et la remise en service après restauration du dump le plus récent.

Ces valeurs sont jugées acceptables au regard du caractère non critique de l'application (un test d'orientation peut être refait, aucune transaction financière n'est engagée). Une procédure de restauration

pas à pas est documentée dans `INSTALL.md`, avec exécution annuelle d'un test de restauration à blanc pour vérifier que les sauvegardes sont effectivement utilisables.

Le code applicatif est sauvegardé par sa présence sur GitHub (origine de référence) et le clone local du serveur. La perte simultanée des deux est traitée comme un événement de très faible probabilité hors périmètre du PCA applicatif.

6.6. Intégration continue

Le pipeline d'intégration continue est défini dans `.github/workflows/lint.yml` et s'exécute à chaque *push* sur les branches `master`, `preprod` et `feature/**`, ainsi qu'à l'ouverture de toute *pull request*. Trois jobs sont actuellement définis :

Job	Outils	Critères de succès
<code>php-lint</code>	<code>php -l</code> sur tous les fichiers <code>*.php</code> .	Aucun fichier en erreur de syntaxe.
<code>forbidden-patterns</code>	<code>grep</code> ciblé sur motifs interdits en production : <code>var_dump</code> , <code>print_r</code> , <code>die</code> , <code>console.log</code> , mots de passe en clair.	Aucune correspondance trouvée.
<code>html-css-sanity</code>	Vérification basique de l'équilibre des balises et des sélecteurs CSS.	Aucun déséquilibre détecté.

Les jobs sont rapides (< 30 secondes) et systématiquement requis avant tout merge sur `master` (politique de protection de branche GitHub). L'absence de framework de test unitaire à ce stade est un choix : la couverture par tests automatisés n'apporterait pas suffisamment de valeur sur un code applicatif aussi linéaire pour justifier la maintenance du harnais. Un palier de tests fonctionnels via *cURL* sur les principales routes API est envisageable en Phase 5.

6.7. Coût d'exploitation et structure budgétaire

6.7.1. Structure des coûts récurrents

L'application n'engage **aucun coût récurrent direct** au-delà des ressources déjà mutualisées par le CNAM PACA. Le tableau ci-dessous récapitule l'origine de chaque ligne d'exploitation.

Ligne d'exploitation	Mode de prise en charge	Coût marginal annuel
Hébergement serveur (CPU, RAM, stockage)	Mutualisé sur l'infrastructure existante	0 €
Bande passante	Mutualisée, volumétrie négligeable (< 200 ko / page, cf. §7.2)	0 €
Base de données MySQL	Instance partagée existante	0 €
Certificat TLS (renouvellement)	Let's Encrypt via <i>certbot</i> (cf. §6.3)	0 €
Domaine <code>r4.ohvenus.fr</code>	Sous-domaine d'un domaine existant	0 €
Sauvegardes	Inclus dans le dispositif mutualisé (§6.5)	0 €
Surveillance externe	Sonde HTTP gratuite type UptimeRobot (§7.4)	0 €
Dépendances logicielles	Aucune (cf. §2.3)	0 €
Total OPEX hors RH		0 €

Aucun coût logiciel ni infrastructure récurrent : conséquence directe des choix de §2.1, et argument poids face à une solution SaaS équivalente.

6.7.2. Charge de maintenance estimée

La maintenance courante est estimée à **environ 1 jour-homme par mois**, couvrant les correctifs de sécurité, l'application des mises à jour mineures de PHP/MySQL/nginx, la revue hebdomadaire des logs (§7.3), la rotation éventuelle des secrets et la surveillance du certificat TLS. Ce volume reste compatible avec une délégation au sein de l'équipe interne du Pôle Innovation, sans recours à un prestataire externe.

Une charge plus élevée (jusqu'à 3 jours par mois) doit être anticipée sur les six premiers mois après mise en production, le temps que les premiers signaux d'usage remontent et que des correctifs ergonomiques soient livrés en réponse aux retours terrain.

6.7.3. Conditions de sortie et coûts associés

La sortie de la solution, par migration vers un autre outil, par arrêt du service ou par changement d'hébergement, n'engage aucun coût contractuel (pas d'abonnement, pas d'engagement de durée). Les seuls coûts à anticiper en cas de sortie seraient :

- Une charge ponctuelle estimée à **2 à 5 jours-homme** pour exporter les données utilisateur (les routes d'export sont prévues, cf. §10.1) et les transférer vers la solution cible.
- Le délai de communication aux utilisateurs et le traitement des éventuelles demandes RGPD résiduelles (suppression définitive, transmission d'archive personnelle).

Aucun coût de pénalité, de licence résiduelle ou de réversibilité fournisseur, contrairement à un déploiement SaaS standard.

7. Performance, sobriété et observabilité

7.1. Indicateurs de performance applicative

Les indicateurs de performance suivants sont définis comme objectifs à respecter en production :

Indicateur	Cible	Mode de mesure	Périodicité
Temps de réponse API médian	< 100 ms	Log nginx (champ <code>\$request_time</code>).	Continu, agrégé hebdomadaire.
Temps de réponse API au 95 ^e percentile	< 300 ms	Idem.	Continu, agrégé hebdomadaire.
TTFB de la page d'accueil	< 200 ms en 4G	Test manuel via Chrome DevTools depuis un terminal mobile.	Mensuel.
Taille transférée par page	< 200 ko (objectif), 400 ko (seuil d'alerte)	Onglet <i>Network</i> du navigateur, mesure à froid.	Avant chaque livraison touchant le front.
Disponibilité mensuelle	≥ 99,5 % (≈ 3,6 h d'indisponibilité tolérée par mois)	Sonde externe HTTP toutes les 5 minutes.	Continu, bilan mensuel.
Temps de chargement complet 4G	< 2 secondes	Test manuel via Chrome DevTools profil 4G.	Mensuel.

Ces indicateurs sont contractuels : tout dépassement sur deux mois consécutifs déclenche une revue technique et, le cas échéant, l'inscription au backlog d'une action corrective.

7.2. Sobriété numérique

Quatre engagements de sobriété numérique, mesurables.

- **Poids transféré** maintenu sous 200 ko par page (cf. tableau §7.1). Toute évolution qui porterait une page au-dessus de 400 ko fait l'objet d'une revue explicite avant merge.
- **Aucune dépendance externe** côté client : pas de polices web (système uniquement), pas de bibliothèques JavaScript tierces, pas d'images bitmap décoratives. Les rares images proviennent du dépôt et sont optimisées en SVG ou en WebP léger.
- **Aucun service externe** appelé côté client (pas d'analytique, pas de *plugins* sociaux, pas de pixel publicitaire). Toute requête sortante d'une page de l'application ira à `r4.ohvenus.fr` exclusivement.

- **Aucune requête côté serveur** vers des services externes (pas de webhook sortant, pas de relais SMTP, pas de récupération d'asset distant). L'application est entièrement autonome.

Vérification : outils de mesure du navigateur (onglet *Network*, *Lighthouse*). La grille de revue avant livraison inclut un contrôle des seuils.

7.3. Journalisation

Les logs de l'application sont produits à trois niveaux distincts.

- **Logs d'accès nginx** (`/var/log/nginx/r4.access.log`) : une ligne par requête, format `combined` étendu avec `$request_time`. Conservation locale 14 jours, rotation quotidienne par `logrotate`.
- **Logs d'erreur PHP** (`/var/log/php/r4.error.log`) : alimenté par `error_log()` du PHP, route `display_errors` désactivée. Toute exception non interceptée par le routeur est enregistrée avec sa trace.
- **Logs applicatifs métier** : événements significatifs (échec de connexion, échec CSRF, accès refusé) écrits sur `STDERR` du processus PHP-FPM, captés par `php-fpm.log`.

Aucun log ne contient de mot de passe ni d'adresse email en clair. Les adresses email apparaissant dans les logs applicatifs sont hachées en SHA-256 tronqué, ce qui permet la corrélation d'événements sans pré-identification.

7.4. Surveillance et alertes

La surveillance active s'articule autour de trois sondes :

1. **Sonde HTTP externe** déclenchée toutes les cinq minutes depuis un service tiers (UptimeRobot ou équivalent gratuit). Vérifie un code 200 sur `https://r4.ohvenus.fr/`. Alerte par mail au chef de projet en cas d'échec consécutif sur deux exécutions.
2. **Sonde de certificat TLS** quotidienne, vérifie la durée résiduelle de validité (cf. §6.3).
3. **Inspection hebdomadaire des logs** par le chef de projet : recherche manuelle de motifs anormaux (pic de 500, pic d'échecs CSRF, requêtes vers des chemins inattendus). Cette inspection alimente le backlog de durcissement le cas échéant.

L'absence d'outil d'observabilité plus poussé (Prometheus, Grafana, ELK) est délibérée à ce stade ; le coût de maintenance de tels outils ne se justifie pas pour une application au trafic encore modeste. Une introduction sera évaluée si la fréquentation atteint plusieurs milliers de tests par mois ou si la criticité du service évolue.

8. Conformité RGPD

8.1. Registre des traitements

L'application met en œuvre **un traitement principal** de données à caractère personnel, déclaré au registre des traitements du Pôle Innovation comme suit :

Rubrique	Contenu
Finalité du traitement	Pré-qualification numérique du profil d'un candidat à la formation et conservation optionnelle de l'historique de tests pour usage personnel.
Base légale (article 6 RGPD)	Consentement explicite (visiteur anonyme et candidat connecté) à l'utilisation du service.
Catégories de données collectées	Identifiant utilisateur, mot de passe haché, adresse email (optionnelle), réponses au test, niveau scolaire déclaré, horodatage.
Catégories de personnes concernées	Visiteurs publics du site, candidats inscrits, administrateurs internes du Pôle Innovation.
Destinataires	Personnel habilité du Pôle Innovation pour l'export agrégé ; aucune transmission à un tiers.
Durée de conservation	24 mois après la dernière connexion pour les comptes utilisateur ; 24 mois après création pour les sessions anonymes (en pratique : aucune session anonyme stockée, cf. §4.1).
Mesures de sécurité	Cf. chapitre 5 du présent document.
Transferts hors UE	Aucun. Hébergement intégral en France.

Le registre est mis à jour à chaque évolution susceptible de l'affecter (ajout de champ, modification de finalité, changement de prestataire). Une copie du registre est transmise au Délégué à la Protection des Données (DPD) de l'établissement avant toute mise en service.

8.2. Privacy by design

Les principes de *privacy by design* du RGPD ont été appliqués dès la conception, et non ajoutés a posteriori.

- **Minimisation**, les données collectées sont strictement nécessaires à la fonction. Aucune adresse postale, aucun numéro de téléphone, aucune date de naissance ne sont demandés.

- **Pseudonymisation**, l'export CSV à destination des Responsables de formation est expurgé du nom d'utilisateur et de l'adresse email. Les lignes sont identifiées par un identifiant numérique stable mais non ré-identifiable hors de la base.
- **Confidentialité par défaut**, un compte nouvellement créé n'expose aucune donnée publique ; les pages d'espace personnel sont strictement réservées au propriétaire.
- **Sécurité de bout en bout**, chiffrement TLS obligatoire en transit (HSTS), mots de passe hachés au repos, sessions durcies (cf. §5.2).
- **Suppression effective**, la cascade `ON DELETE CASCADE` garantit qu'une suppression de compte purge l'intégralité des données associées en une seule opération atomique.

8.3. Droits des personnes concernées

Les droits prévus par le RGPD sont implémentés ou cadrés comme suit.

Droit	Implémentation actuelle	Délai cible
Droit d'accès	Page <code>account.html</code> accessible par l'utilisateur : ses tests passés, son niveau, ses recommandations sont consultables.	Immédiat.
Droit de rectification	Pour le moment hors application : seul le mot de passe est modifiable depuis l'interface. Toute autre rectification se fait sur demande au chef de projet.	5 jours ouvrés.
Droit à l'effacement (« droit à l'oubli »)	Suppression du compte sur demande, déclenchant la cascade SQL. Une route <code>DELETE /api/me</code> est en backlog Phase 5 pour permettre la suppression auto-service.	30 jours.
Droit à la portabilité	Export JSON des données du compte, sur demande au chef de projet. Une route <code>GET /api/me/export</code> est en backlog Phase 5.	30 jours.
Droit d'opposition	Désactivation du compte sur demande ; données conservées en veille 24 mois puis purgées.	5 jours ouvrés.
Droit de réclamation à la CNIL	Information explicite sur la page de mention légale.	Sans objet (tiers).

Les délais cibles sont alignés sur les obligations RGPD (un mois maximum, un mois de prolongation possible si la demande est complexe). Toute demande reçue par mail à l'adresse de contact du DPD est consignée et tracée.

8.4. Mentions légales et information

Une page de mentions légales et une page de politique de confidentialité sont à intégrer avant la mise en service publique du domaine de production. Elles précisent : l'identité du responsable de traitement, le DPD de l'établissement, les finalités du traitement, les bases légales, les durées de conservation, les destinataires, les droits des personnes et les voies de recours. Ces deux pages sont rédigées par le chef de projet en collaboration avec le DPD et figurent en backlog de Phase 4.

9. Accessibilité numérique

9.1. Référentiel et niveau visé

L'application vise la conformité au **référentiel WCAG 2.1 niveau AA**, qui constitue l'attendu courant des organismes publics français en application du Référentiel Général d'Amélioration de l'Accessibilité (RGAA). Cette conformité n'est pas purement déclarative : un audit interne est conduit par le chef de projet avant chaque livraison majeure, complété par une vérification automatisée systématique.

Les quatre principes du référentiel sont traités explicitement.

- **Perceptibilité**, contrastes vérifiés au-dessus de 4,5:1 pour le texte courant et 3:1 pour les éléments d'interface, taille de texte minimale de 16 pixels sur mobile, possibilité de zoom à 200 % sans perte de fonctionnalité.
- **Utilisabilité**, navigation au clavier sur tous les parcours principaux, focus visible (anneau bleu), aucune action déclenchée par le seul mouvement de la souris, temporisations désactivables.
- **Compréhensibilité**, interface uniformément en français, niveau de lecture B2 visé, libellés clairs sans jargon administratif, messages d'erreur explicites avec proposition d'action corrective.
- **Robustesse**, code HTML5 valide, attributs ARIA utilisés à bon escient (boutons `aria-label` sur le bouton « Retour », champs de formulaire associés à leur étiquette, *progress bar* avec `role`, `aria-valuemin`, `aria-valuemax`, `aria-valuenow`).

9.2. Outils de contrôle

Le contrôle d'accessibilité s'effectue par combinaison d'outils automatisés et de tests manuels.

- **Audit automatisé** par l'extension *axe-core* du navigateur, exécutée sur chaque page principale avant livraison. Tout *issue* de niveau *critical* ou *serious* est résolu avant merge ; les niveaux *moderate* et *minor* alimentent le backlog.
- **Vérification de contraste** par l'inspecteur de contraste intégré à Firefox sur les composants stylés.
- **Test au clavier** systématique : démarrage du test, navigation entre vues, soumission de réponses, accès au compte, tous sans la souris.
- **Test au lecteur d'écran** : passages réguliers avec NVDA (sous Windows en machine virtuelle) sur les écrans de connexion, de question et de résultat.
- **Test en conditions de vision réduite** : simulation de daltonisme via les filtres natifs de Chrome DevTools, pour vérifier que la signalétique ne repose pas exclusivement sur la couleur.

9.3. Limites connues

Les non-conformités identifiées sont explicitement répertoriées dans un backlog d'accessibilité. Au moment de la rédaction, deux limitations sont signalées :

1. La modale « Connectez-vous pour sauvegarder vos résultats » s'affiche en bas du parcours sans piéger le focus dans un cycle restreint, ce qui peut perturber la lecture au clavier. Correction prévue en Phase 5.
2. Les boutons d'option du test ne disposent pas encore d'un `aria-pressed` reflétant leur état après sélection. Correction prévue en Phase 5.

Ces limitations sont consignées et n'invalident pas la déclaration de conformité globale niveau AA, dans la mesure où elles concernent un nombre restreint de composants et où le parcours principal reste navigable.

10. Plan d'évolution technique

10.1. Backlog technique de Phase 5

La Phase 5, postérieure à la livraison école, intègre déjà un ensemble d'évolutions techniques planifiées. Trois axes sont identifiés.

Axe sécurité et conformité :

- Externalisation des `style=""` résiduels pour permettre une CSP stricte sans `'unsafe-inline'`.
- Implémentation d'un `aria-pressed` sur les boutons d'option du test.
- Route `DELETE /api/me` pour l'auto-service du droit à l'effacement.
- Route `GET /api/me/export` pour l'auto-service du droit à la portabilité.

Axe ergonomie :

- Bouton « Retour » dans le parcours questionnaire (livré début mai 2026, première itération de cet axe).
- Mémorisation locale des réponses partielles pour permettre la reprise après actualisation accidentelle.
- Page catalogue indépendante du test, déjà accessible aux utilisateurs connectés, à étendre aux visiteurs anonymes.

Axe observabilité :

- Mise en place d'un export Prometheus minimal (compteurs HTTP, durée des requêtes) si le volume justifie l'investissement.
- Tableau Grafana embarqué sur le poste du chef de projet pour les indicateurs de §7.1.
- Sonde de cohérence base/code (vérification des invariants d'intégrité référentielle au-delà de ce que MySQL impose).

10.2. Évolutions hors v1 envisagées

Plusieurs évolutions ont été délibérément écartées du périmètre v1 mais figurent à la prospective technique.

- **Single Sign-On établissement**, intégration au futur annuaire SSO du CNAM PACA, à condition que ce dernier soit ouvert aux applications du Pôle Innovation. Réduirait la dette d'authentification locale.
- **Internationalisation**, extraction des chaînes en fichiers de langue, mécanisme de bascule via préférence utilisateur. À envisager si une demande s'exprime au-delà du périmètre francophone.

- **Application mobile native**, non envisagée à court terme, la version *responsive* étant jugée suffisante. Une *Progressive Web App* (PWA) constitue une étape intermédiaire envisageable à coût modéré (manifest, service worker minimal, prompt d'installation).
- **Statistiques publiques agrégées**, *dashboard* institutionnel présentant les tendances du test (formations les plus recommandées, profil des candidats). Suppose une revue d'opportunité côté Communication et un travail de pseudonymisation supplémentaire.
- **Revue annuelle du scoring**, engagement pris dans le CCF (§ Annexe B) d'organiser une revue conjointe avec les Responsables de formation pour ajuster la pondération des trente questions vis-à-vis des dix-huit formations. Le compte rendu de cette revue sera versionné dans `docs/cr/` (réunion type M3 récurrente annuelle), avec migration SQL associée si la pondération évolue.

10.3. Critères de bascule vers une refonte technique

Une refonte de fond (framework PHP, séparation API / front, observabilité poussée) ne se justifierait que si plusieurs seuils suivants étaient durablement franchis :

- Plus de cinq mille tests réalisés par mois, mesuré sur trois mois consécutifs.
- Plus de cinquante administrateurs distincts, ce qui complexifierait significativement la gestion des autorisations.
- Demandes fonctionnelles lourdes incompatibles avec l'architecture actuelle (workflow multi-étapes, intégration temps réel à la plateforme d'inscription, internationalisation de l'administration).
- Charge opérationnelle de maintenance dépassant un demi-équivalent temps plein.

Aucun de ces seuils n'est atteint à ce jour. La pile actuelle reste donc le bon outil pour le besoin tel qu'il est exprimé.

11. Critères d'acceptation technique

11.1. Critères généraux

La solution est techniquement réceptionnable lorsque l'ensemble des critères ci-dessous sont satisfaits, vérifiés conjointement par le chef de projet et le Maître d'Apprentissage en revue technique formelle.

Critère	Vérification
L'application est servie sous HTTPS, certificat valide plus de 30 jours.	<code>openssl s_client</code> ou inspection navigateur.
Les en-têtes de sécurité (cf. §5.5) sont présents sur toutes les réponses.	<code>curl -I</code> sur la racine.
Les routes <code>admin/*</code> retournent 403 en l'absence d'authentification administrateur.	Test manuel sans session, puis avec session utilisateur non admin.
Toute route mutante refusée sans jeton CSRF valide.	Test manuel via <code>curl -X POST</code> sans en-tête, puis avec en-tête erroné.
Tentative répétée sur <code>/api/auth</code> plafonnée à cinq échecs en cinq minutes.	Test manuel via boucle <code>curl</code> .
<code>display_errors</code> désactivé en production, aucune trace technique fuit dans les réponses 500.	Test manuel via requête déclenchant une erreur (route inexistante, payload invalide).
Aucune dépendance externe applicative présente dans le dépôt.	Inspection manuelle (pas de <code>vendor/</code> , pas de <code>node_modules/</code> , pas de <code>composer.json</code>).
L'export CSV contient les colonnes attendues, sans information nominative non anonymisée.	Lancement manuel de l'export depuis l'interface admin.
La cascade <code>ON DELETE CASCADE</code> purge effectivement les réponses lors de la suppression d'un utilisateur.	Test SQL ad hoc sur la base de pré-production.
Le parcours utilisateur principal s'effectue sans erreur sur quatre profils d'appareil (smartphone iOS, smartphone Android, tablette, desktop).	Test manuel par le chef de projet et un testeur tiers.
Le poids transféré pour la page d'accueil reste inférieur à 200 ko.	Onglet <i>Network</i> du navigateur, première visite à froid.
Le rapport <i>axe-core</i> ne signale aucune issue <i>critical</i> ou <i>serious</i> .	Extension <i>axe-core</i> exécutée sur les pages principales.

11.2. Critères de mise en production

Au-delà des critères techniques généraux, la mise en production effective n'est prononcée qu'après :

- Validation explicite par le Maître d'Apprentissage du dossier de mise en service (vhost configuré, base initialisée, migrations appliquées, sauvegardes opérationnelles).
- Validation par le DPD de la conformité RGPD (registre à jour, mentions légales en ligne, procédure de droits d'accès opérationnelle).
- Réalisation d'un test de bascule (simulation d'arrêt et de redémarrage du service) avec mesure du temps de remise en service.
- Communication aux services Communication et Hub de la disponibilité de l'outil et des modalités d'incident.

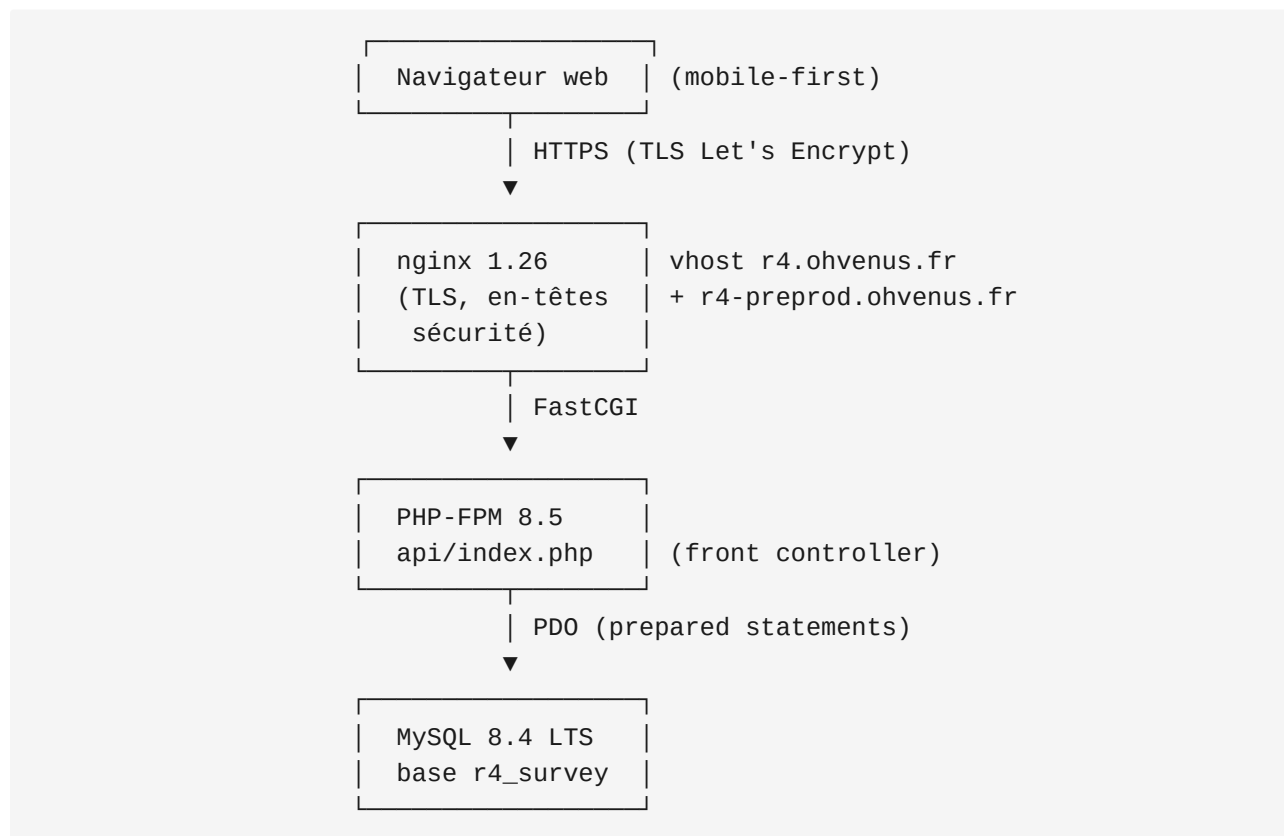
11.3. Critères de réversibilité

La réversibilité de la solution est elle-même un critère technique : à tout moment, il doit être possible pour le commanditaire de se passer du chef de projet sortant. Cette réversibilité est garantie par :

- L'intégralité du code applicatif sur GitHub, accessible à tout détenteur d'un compte ayant les droits.
- La documentation `INSTALL.md` reproduisant pas à pas l'installation sur une nouvelle machine.
- L'absence de secret hors `.env` (lui-même décrit dans `INSTALL.md`).
- Le respect strict des conventions de pile (PHP, MySQL, nginx) qui rendent la solution opérable par tout administrateur système familier de cette pile.
- L'absence de dépendance à un service externe payant ou nominatif (pas d'abonnement, pas de compte personnel rattaché à un service tiers).

12. Annexes

Annexe A, Diagramme d'architecture (textuel)

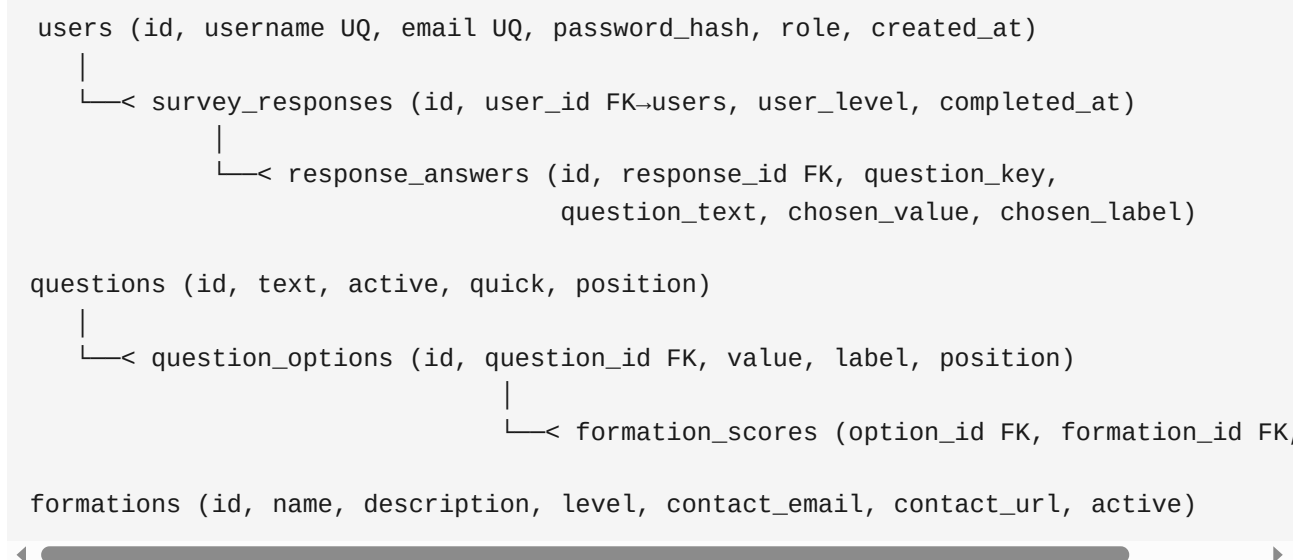


Annexe B, Inventaire des routes API

Méthode	Route	Authentification	Description
GET	/api/csrf	Aucune	Génération / lecture du jeton CSRF de session.
GET	/api/auth	Aucune	État de la session courante (anonyme ou authentifiée).
POST	/api/auth	CSRF	Inscription, connexion ou déconnexion selon action .
GET	/api/questions?mode=quick\ full	Aucune	Liste des questions actives, filtrées selon le mode.
GET	/api/formations	Aucune	Catalogue public des 18 formations.
POST	/api/recommend	Aucune	Calcul de recommandation à la volée.
GET	/api/answers	Session utilisateur	Dernière session de réponses de l'utilisateur.
POST	/api/answers	Session utilisateur + CSRF	Persistance d'une session de réponses.
GET	/api/me/tests	Session utilisateur	Historique complet des tests passés par l'utilisateur.
GET	/api/stats	Session admin	Statistiques agrégées sur les réponses.
GET	/api/admin/questions	Session admin	Liste complète des questions (actives + inactives).
POST	/api/admin/questions	Session admin + CSRF	Création d'une question.
PUT	/api/admin/questions	Session admin + CSRF	Modification d'une question.
DELETE	/api/admin/questions	Session admin + CSRF	Désactivation logique d'une question.
GET	/api/admin/formations	Session admin	Liste complète des formations.

Méthode	Route	Authentification	Description
POST	/api/admin/formations	Session admin + CSRF	Création d'une formation.
PUT	/api/admin/formations	Session admin + CSRF	Modification d'une formation.
DELETE	/api/admin/formations	Session admin + CSRF	Désactivation logique d'une formation.
GET	/api/admin/export	Session admin	Export CSV de l'historique des réponses.

Annexe C, Schéma de la base de données (résumé)



Annexe D, Extrait du fichier `.env` de référence

```

# Base de données
DB_HOST=localhost
DB_NAME=r4_survey
DB_USER=r4_user
DB_PASS=<secret, géré via pass>
DB_CHARSET=utf8mb4

# Application
APP_ENV=production
APP_URL=https://r4.ohvenus.fr

```

Annexe E, Politique de gestion des incidents

Niveau	Description	Délai d'intervention	Communication
P1	Service totalement indisponible.	< 4 heures (RTO).	Information immédiate au MA + DPD si fuite suspectée.
P2	Fonctionnalité majeure dégradée (test inopérant, admin inaccessible).	< 1 jour ouvré.	Information au MA.
P3	Anomalie mineure (affichage dégradé, lien cassé).	Prochaine livraison.	Inscription au backlog.
P4	Demande d'évolution.	Backlog standard.	Issue GitHub.

Tout incident de niveau P1 ou P2 fait l'objet d'un compte rendu écrit consigné dans `docs/incidents/` après résolution, indiquant chronologie, cause racine, mesures correctives et éventuelles mesures préventives à intégrer au backlog.

Annexe F, Liste des migrations SQL appliquées

Fichier	Date d'application preprod	Date d'application prod	Auteur	Effet principal
schema.sql	2026-03-16	2026-04-28	A. Demoisson	Création initiale des trois tables transactionnelles (users , survey_responses , response_answers).
migration_v2.sql	2026-04-20	2026-04-28	A. Demoisson	Création des quatre tables de référence (questions , question_options , formations , formation_scores) + CRUD admin.
migration_v3_cnam.sql	2026-04-28	2026-04-28	A. Demoisson	Périmètre 18 formations CNAM PACA + niveau RNCP.
migration_v4_questions.sql	2026-04-28	2026-04-28	A. Demoisson	Quiz étendu à 30 questions thématiques.
migration_v5_quick.sql	2026-04-28	2026-04-28	A. Demoisson	Mode test rapide (10 questions).